

H0004 — Cargar competidores

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo)

Épica: Inscripción y llaves

3 puntos

KANO: Básico

Rol: Coordinador

1. Qué se implementó

El **Coordinador** puede **cargar competidores** en las categorías de un torneo y **ver el listado** de inscriptos. Cada competidor lleva sus datos personales, físicos, de escuela y responsable, y queda asociado a una categoría.

Sin entidad ni migración nuevas: la entidad `Competidor`, su configuración EF y la tabla `competidores` ya existían. Faltaba el `CompetidorRepository` (no estaba implementado); se creó y registró en DI. El resto es Application + API + frontend.

2. Backend (Clean Architecture + CQRS)

Archivo	Rol
Features/Competidores/CompetidorResponse.cs	DTO con <code>nombreCompleto</code> y <code>categoriaNombre</code> ya resueltos.
CompetidorMapping.cs	Helper: <code>.Adapt<CompetidorResponse>()</code> (Mapster) + <code>with</code> para los campos calculados.
ICompetidorData.cs + CompetidorValidationRules.cs	Interfaz de campos comunes + reglas de validación compartidas (evita duplicar entre command y request).
Commands/CreateCompetidor/...	Command + Handler + Validator del alta.
Queries/GetCompetidoresByTorneo/...	Query + Handler del listado.
Infrastructure/.../CompetidorRepository.cs	EF Core con <code>Include(Categoria)</code> para traer el nombre de la categoría.

Método	Ruta	Rol
GET	<code>/api/v1/torneos/{torneoId}/competidores</code>	Coordinador
POST	<code>/api/v1/torneos/{torneoId}/competidores</code>	Coordinador

Reglas de negocio en el alta.

- El torneo debe existir (`NotFoundException` → 404).
- El torneo no puede estar `Finalizado` (`ConflictException` → 409).
- La categoría debe existir y *pertenecer a ese torneo* (si no → 404), evitando cargar un competidor en una categoría de otro torneo.

Validación de datos (FluentValidation)

- Nombre/Apellido requeridos (máx. 150); Escuela/Responsable requeridos (máx. 200); Teléfono opcional.
- Edad entre 1 y 120; Peso > 0 y ≤ 500 kg; Altura > 0 y ≤ 3 m.
- Graduación: debe ser una del enum oficial de cinturones.

Decisión de alcance: **no se valida (todavía) que el competidor “encaje” en los rangos de la categoría** (edad/peso/graduación). Se dejó como responsabilidad del Coordinador; además el competidor no tiene campo sexo, por lo que ese cruce sería parcial. Es un candidato claro para una mejora futura.

Flujo de una carga

```
POST /torneos/{id}/competidores (rol Coordinador)
-> CreateCompetidorRequestValidator (valida datos del competidor)
-> CreateCompetidorEndpoint (arma el Command y lo envia con MediatR)
-> ValidationBehavior (pipeline: valida el Command)
-> CreateCompetidorCommandHandler (torneo 404/409, categoria 404, persiste)
-> CompetidorRepository.AddAsync (EF Core -> PostgreSQL)
-> 201 Created + CompetidorResponse
```

3. Frontend (React 19 + TanStack Query)

Archivo	Rol
types/index.ts	Competidor y CreateCompetidorInput .
lib/api.ts	competidores.listar/cargar .
hooks/useCompetidores.ts	Query keys por torneo; useCompetidores y useCargarCompetidor (invalida la lista en onSuccess).
components/competidores/CompetidorForm.tsx	Alta (RHF + Zod) con select de categoría ; resetea para cargar varios seguidos.
components/competidores/CompetidorList.tsx	Tabla semántica responsive (scroll horizontal en mobile) con estados de carga/error/vacío.
routes/torneos/\$torneoid/competidores.tsx	Página (solo Coordinador); si no hay categorías, invita a crearlas primero.
App.tsx + TorneoCard.tsx	Ruta protegida con RoleGuard(['Coordinador']) + enlace “Competidores →” en la card.

4. Decisiones de diseño

Nombre de categoría resuelto en el backend. El listado hace Include(Categoria) y la respuesta ya trae categoriaNombre , así el frontend muestra la categoría sin pedir datos extra ni cruzar en el cliente.

Guía cuando faltan categorías. Un competidor necesita una categoría; si el torneo no tiene ninguna, la página no muestra el formulario y enlaza directo a “crear una categoría”. Previene el error antes de que ocurra.

Reglas compartidas desde el inicio. Siguiendo lo aprendido en categorías, la validación de competidor vive en `CompetidorValidationRules` sobre `ICompetidorData`, reutilizada por el validator del comando y el del request (sin duplicar).

5. Pruebas

- `CreateCompetidorCommandHandlerTests` — alta válida (con nombre completo y categoría), torneo inexistente (404), torneo finalizado (409), categoría inexistente (404) y categoría de otro torneo (404).
- `CreateCompetidorCommandValidatorTests` — nombre vacío, categoría vacía, edad fuera de rango, graduación inválida, peso inválido y altura fuera de rango.

✓ Backend compila 0 errores · **56/56 tests** en verde (+12).

✓ Frontend `tsc + vite build` sin errores.

Nota: la verificación end-to-end contra la API quedó pendiente porque el entorno Docker/backend estaba detenido. El backend compila y sus tests pasan; al levantar el entorno queda listo para probar (cargar competidor → verlo en la tabla).

6. Checklist

Ítem	Estado
Command + Handler + Validator (alta)	✓
Query + Handler (listado)	✓
Repositorio + registro en DI	✓
Endpoints POST/GET con Roles("Coordinador")	✓
Reglas: torneo 404/409 · categoría 404 y pertenencia	✓
Mapeo Mapster (.Adapt + with)	✓
Tests handler + validator	✓ 12 nuevos
Types + api + hook TanStack Query	✓
Form (select categoría) + tabla + ruta + enlace	✓ responsive, WCAG 2.2 AA
Comentarios XML <code><summary></code> en el código nuevo	✓
Verificación E2E	pendiente (entorno caído)

